

LINKED LIST

PROBLEM 4

QUESTION:

DS	Session	Linked List	Question Information	Level 1	Challenge 34
Question description Problem Description: For Engineering Second -year students, the instructor is organising a surprise test. He's given you a new node is added before the given node of the given Linked List. For example if the given Linked List is 5->10->15->20->25 and we add an item 30 before node 15, then the Linked List becomes 5->10->30->15->20->25. Since a Linked List is typically represented by the head of it, we have to traverse the list till node and then insert the node. Constraints: 1 < arr <100 Input Format First line contains the number of datas- N. Second line contains N integers(the given linked list). Third line contains the node P before which the node to be inserted. Fourth line contain the node X to be inserted. Output Format case 1(node P found in Linked list) :					

Display the final Linked List.

case 2(node P not found in Linked list) :

Print Node not found!

Display the Linked list.

Logical Test Cases

Test Case 1	Test Case 2
INPUT (STDIN) 4 115 210 116 110 116 17 EXPECTED OUTPUT Linked List:->115->210->17->116->110	INPUT (STDIN) 4 15 20 16 10 25 17 EXPECTED OUTPUT Node not found! Linked List:->15->20->16->10

Mandatory Test Cases

Test Case 1	Test Case 2	Test Case 3
KEYWORD struct node	KEYWORD node *next;	KEYWORD *start
Test Case 4	Test Case 5	
KEYWORD p2->next=p1;	KEYWORD void display()	

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

3

Test Case 2

TOKEN COUNT

275

Test Case 3

NLOC

66

SOURCE CODE:

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *next;
};

struct node *start = NULL;

void display() {
    printf("Linked List:");
    struct node *temp = start;
    while (temp != NULL) {
        printf("->%d", temp->data);
        temp = temp->next;
    }
    printf("\n");
}

int main() {
    int n, val, p, X;
    if (scanf("%d", &n) != 1) return 0;
    struct node *last = NULL;
    for (int i = 0; i < n; i++) {
        scanf("%d", &val);
        struct node *new_node = (struct node*)malloc(sizeof(struct node));
        new_node->data = val;
        new_node->next = NULL;
        if (start == NULL) {
            start = new_node;
        } else {
            last->next = new_node;
        }
        last = new_node;
    }
    scanf("%d", &p);
    scanf("%d", &X);

    struct node *p1 = (struct node*)malloc(sizeof(struct node));
    p1->data = X;
    p1->next = NULL;

    struct node *curr = start;
    struct node *p2 = NULL;
    int found = 0;

    if (start != NULL && start->data == p) {
        p1->next = start;
        start = p1;
        found = 1;
    } else {
        while (curr != NULL) {
            if (curr->data == p) {
                found = 1;
                p2->next = p1;
                p1->next = curr;
                break;
            }
            p2 = curr;
            curr = curr->next;
        }
    }

    if (!found) {
        printf("Node not found!\n");
        free(p1);
    }

    display();
    return 0;
}
```

OUTPUT:

```
4
115 210 116 110
116
17
Linked List:->115->210->17->116->110
```

```
4
15 20 16 10
25
17
Node not found!
Linked List:->15->20->16->10
```